PROYECTO INTEGRADOR

Cuarto Semestre – Segunda Entrega

Prototipo de Aplicación de Escritorio para la Gestión de Grabadores Externos de Video Profesionales

Integrantes:

[Garcia]

[estebam]

[ajumi]

Docente:

Gabriel Perez Moreno

Asignatura: Programación 2 **Grupo:** [411]

Institución Universitaria Antonio José Camacho

Programa de Tecnología en Sistemas de Información (PTSI)

Santiago de Cali, 2026

Contenido

Lista de figuras **3**
Lista de tablas **4**
1. Planteamiento del problema **5**
1.1 Planteamiento del problema 5
2. Objetivos **6**
2.1 Objetivo general 6
2.2 Objetivos específicos 6
3. Justificación **7**
4. Metodología **8**
4.1 Enfoque de desarrollo 8
4.2 Herramientas tecnológicas 8
4.3 Fases y actividades 8
5. Especificación de requerimientos funcionales y no funcionales **10**
5.1 Requerimientos funcionales 10
5.2 Requerimientos no funcionales 11
5.3 Trazabilidad de requerimientos a la implementación 12
6. Diagrama de casos de uso **13**
7. Diagrama de clases **15**
7.1 Diagrama de clases del sistema 15
7.2 Descripción de las clases 16
7.3 Atención a las observaciones del docente 17
7.4 Arquitectura MVC 18
7.5 Implementación de las clases, la conexión y los DAO 19
7.6 Interfaz gráfica de usuario 20
8. Diagramas de actividades **22**
8.1 Inicio de sesión 22
8.2 Registro de un grabador con inventario inicial 22
8.3 Registro de un movimiento de inventario 23
9. Diagrama entidad-relación **25**
10. Diagrama de datos **26**
11. Mockup **28**
Referencias **31**
Anexos **32**
Anexo A. Guion de entrevista para el levantamiento de requerimientos 32
Anexo B. Script SQL de la base de datos 33
Anexo C. Estructura de paquetes del proyecto 35

Lista de figuras

Figura 1. Diagrama de casos de uso del sistema....13
Figura 2. Diagrama de clases del sistema de gestión de grabadores externos de video....15
Figura 3. Enumeraciones del dominio....16
Figura 4. Arquitectura MVC de la aplicación por paquetes....19
Figura 5. Diagrama de actividades del inicio de sesión....22
Figura 6. Diagrama de actividades del registro de un grabador....23
Figura 7. Diagrama de actividades del registro de un movimiento de inventario....24
Figura 8. Diagrama entidad-relación del sistema....25
Figura 9. Diagrama de datos (modelo relacional) del sistema....26
Figura 10. Maqueta de la ventana de inicio de sesión....28
Figura 11. Maqueta del módulo de grabadores....29
Figura 12. Maqueta del módulo de inventario y movimientos....29
Figura 13. Maqueta del módulo de generación de reportes e historial de reportes....30

Lista de tablas

Tabla 1. Herramientas utilizadas en el proyecto....8
Tabla 2. Fases, actividades y productos del proyecto....8
Tabla 3. Requerimientos funcionales....10
Tabla 4. Requerimientos no funcionales....11
Tabla 5. Trazabilidad entre requerimientos funcionales y código....12
Tabla 6. Descripción de los casos de uso....13
Tabla 7. Clases del modelo y su responsabilidad....16
Tabla 8. Observaciones de la primera entrega y decisión tomada....17
Tabla 9. Componentes de la interfaz por módulo....21
Tabla 10. Restricciones de integridad de la base de datos....27
Tabla 11. Paquetes y archivos del código fuente....35

1. Planteamiento del problema

1.1 Planteamiento del problema

En la industria audiovisual profesional, los grabadores externos de video son dispositivos fundamentales para mejorar la calidad de grabación y optimizar los flujos de trabajo. Sin embargo, no existe una herramienta sencilla que permita gestionar de manera organizada la información técnica, operativa y de inventario de estos equipos. La información suele estar dispersa en catálogos, documentos, sitios web y registros manuales, lo que dificulta la consulta de especificaciones, la comparación entre modelos, la verificación de compatibilidad con cámaras y el control de disponibilidad.

Existen plataformas que ofrecen información técnica sobre grabadores externos de video, como los sitios oficiales de fabricantes (Atomos, s. f.; Blackmagic Design, s. f.; Video Devices, s. f.) y tiendas especializadas en equipos audiovisuales. Estas herramientas permiten consultar especificaciones, precios y compatibilidad, pero no están diseñadas para gestionar inventarios internos ni para centralizar información operativa. Por otro lado, algunos sistemas de inventario audiovisual permiten registrar equipos, pero son genéricos y no contemplan características específicas de los grabadores externos, como formatos de grabación, conexiones, compatibilidad o accesorios.

Esta falta de centralización genera errores, pérdida de información y decisiones técnicas ineficientes. Por ello, se propone el desarrollo de un sistema de información que permita centralizar y administrar la información relacionada con grabadores externos de video profesionales, construido sobre la base del prototipo elaborado en el semestre anterior y ampliado con los conocimientos de Programación 2, Bases de Datos 2 e Ingeniería de Software 1, para dar como resultado una aplicación funcional completa.

Pregunta de investigación: ¿Cómo puede un sistema de información de escritorio, con base de datos relacional, centralizar y facilitar la gestión técnica, de compatibilidad y de inventario de los grabadores externos de video profesionales, mejorando la eficiencia y confiabilidad frente al manejo disperso actual?

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio en Java, con arquitectura MVC y base de datos relacional, que permita gestionar la información técnica, la compatibilidad y el inventario de grabadores externos de video, con trazabilidad de los movimientos y control de acceso por roles.

2.2 Objetivos específicos

1. Especificar los requerimientos funcionales y no funcionales del sistema y modelarlos mediante diagramas UML de casos de uso, clases y actividades.
2. Implementar el registro, la consulta, la edición y la desactivación lógica de grabadores externos, con búsqueda por marca, modelo, formato, resolución y disponibilidad.
3. Implementar la administración de inventario con movimientos de entrada, salida, ajuste y cambio de estado, ejecutados en transacciones y registrados en un historial auditable.
4. Implementar la gestión de cámaras, accesorios, imágenes de referencia y compatibilidad entre grabadores y cámaras.
5. Implementar la autenticación y el control de acceso para los roles Administrador y Consultor, aplicando herencia y polimorfismo.
6. Construir una base de datos relacional SQLite conectada mediante JDBC y consultas parametrizadas, siguiendo el patrón de capas Modelo, Vista, Controlador y DAO.
7. Generar reportes que presenten el análisis de la información almacenada.

3. Justificación

Los grabadores externos son equipos de alto valor técnico y económico, por lo que su disponibilidad y su correcta selección influyen directamente en la calidad y el costo de una producción. Centralizar su información reduce el tiempo de consulta, evita errores por registros manuales y permite decidir con datos sobre compras, mantenimiento y asignación de equipos.

Los beneficios esperados son: (a) mayor eficiencia operativa, porque una sola aplicación reemplaza la consulta de catálogos y hojas dispersas; (b) menos errores, ya que las operaciones de inventario se validan (por ejemplo, no se permite una salida mayor al stock) y se ejecutan de forma atómica; (c) trazabilidad, pues cada movimiento queda con usuario, fecha, stock y estado anterior y nuevo; y (d) apoyo a la decisión técnica mediante el comparador y la verificación de compatibilidad con cámaras.

El proyecto es viable técnicamente porque se desarrolla con herramientas gratuitas y multiplataforma (Java, Swing, SQLite y Git), no requiere servidor de base de datos y parte del prototipo ya validado en el semestre anterior. Desde lo académico, integra la programación orientada a objetos, el acceso a datos con JDBC, el modelado UML y el diseño de bases de datos relacionales (Deitel y Deitel, 2017; Date, 2004). La aceptación por parte de los futuros usuarios se evalúa con la entrevista del caso de estudio descrita en el Anexo A.

4. Metodología

4.1 Enfoque de desarrollo

Se emplea un desarrollo de prototipado incremental: el sistema se construye por módulos (autenticación, grabadores, inventario, cámaras, compatibilidad, accesorios, imágenes y reportes) y cada incremento se revisa antes de iniciar el siguiente (Pressman, 2010; Sommerville, 2011). Los artefactos de diseño se elaboran antes de codificar, de modo que el código implemente lo modelado.

4.2 Herramientas tecnológicas

Tabla 1

Herramientas utilizadas en el proyecto

Componente	Herramienta	Uso
Lenguaje	Java 17	Implementación de las clases, controladores y DAOs
Interfaz gráfica	Java Swing	Ventanas, paneles, tablas y listas desplegables (Oracle, s. f.)
Base de datos	SQLite con JDBC	Persistencia local en un único archivo, sin servidor (SQLite, s. f.-a)
Editor	Visual Studio Code	Desarrollo y depuración del código Java
Control de versiones	Git y GitHub	Trabajo colaborativo del equipo
Diagramas y mockups	Graphviz, Draw.io y herramientas de dibujo	UML, modelo entidad-relación y maquetas de interfaz

Nota. Elaboración propia.

4.3 Fases y actividades

Tabla 2

Fases, actividades y productos del proyecto

Fase	Actividades	Producto
1. Levantamiento	Visita al caso de estudio, entrevista, revisión del prototipo anterior y de fichas técnicas.	Tabla de requerimientos numerados
2. Modelado y diseño	Casos de uso, diagrama de clases, diagramas de actividades, modelo entidad-relación, diagrama de datos y mockups.	Diagramas UML y maquetas

Fase	Actividades	Producto
3. Implementación	Creación del esquema SQLite, clases del modelo, DAOs con JDBC, controladores y paneles Swing, módulo por módulo.	Aplicación funcional
4. Pruebas	Pruebas funcionales de CRUD, filtros, transacciones de inventario, roles y reportes.	Casos de prueba y correcciones
5. Documentación y sustentación	Consolidación del documento bajo normas APA, ajuste del código y preparación de la presentación.	Documento final y sustentación

Nota. Elaboración propia.

En la primera entrega se completaron las fases 1 y 2 en su versión inicial. En esta segunda entrega se corrigieron los diseños según las observaciones del docente (sección 7.3) y se documentó la implementación de la fase 3.

5. Especificación de requerimientos funcionales y no funcionales

Los requerimientos se numeran para poder rastrearlos en los casos de uso, en el código y en las pruebas. La numeración de los requerimientos RF05 (desactivación lógica) y RF06 (búsqueda) coincide con la usada en los comentarios del código fuente. Los roles son Administrador (A) y Consultor (C).

5.1 Requerimientos funcionales

Tabla 3

Requerimientos funcionales

ID	Descripción	Rol
RF01	El sistema debe permitir registrar un grabador externo con marca, modelo, número de serie, año, formatos, resolución, conexiones, almacenamiento y precio.	A
RF02	El sistema debe listar los grabadores activos ordenados por marca y modelo.	A, C
RF03	El sistema debe mostrar la ficha técnica de un grabador con sus imágenes, accesorios y cámaras compatibles.	A, C
RF04	El sistema debe permitir editar los datos de un grabador existente.	A
RF05	El sistema debe desactivar un grabador mediante eliminación lógica, sin borrar su historial.	A
RF06	El sistema debe buscar y filtrar grabadores por término general, marca, modelo, formato, resolución y disponibilidad.	A, C
RF07	El sistema debe comparar las especificaciones de varios grabadores.	A, C
RF08	El sistema debe registrar, editar, listar y desactivar cámaras.	A
RF09	El sistema debe registrar la compatibilidad entre un grabador y una cámara indicando el tipo de conexión y el nivel (total, parcial o no compatible).	A
RF10	El sistema debe gestionar los accesorios de cada grabador con nombre, descripción, cantidad y estado.	A
RF11	El sistema debe asociar hasta tres imágenes de referencia por grabador (frontal, posterior y lateral).	A
RF12	El sistema debe crear el registro de inventario inicial (stock y estado) al registrar un grabador, en la misma transacción.	A
RF13	El sistema debe registrar movimientos de inventario de tipo entrada, salida, ajuste y cambio de estado, validando que el stock no quede negativo.	A
RF14	El sistema debe permitir actualizar el stock y el estado de un inventario, dejando constancia en el historial.	A
RF15	El sistema debe consultar el historial de movimientos filtrando por rango de fechas, tipo y usuario.	A, C
RF16	El sistema debe generar reportes con la información almacenada.	A, C

ID	Descripción	Rol
RF17	El sistema debe autenticar al usuario con correo y contraseña antes de permitir el acceso.	A, C
RF18	El sistema debe permitir crear, editar y desactivar usuarios y asignarles un rol.	A
RF19	El sistema debe restringir las funciones según el rol: el Consultor solo puede consultar.	A, C

Nota. Elaboración propia.

5.2 Requerimientos no funcionales

Los requerimientos no funcionales se clasifican según las características de calidad de la norma ISO/IEC 25010 (International Organization for Standardization [ISO] e International Electrotechnical Commission [IEC], 2011).

Tabla 4

Requerimientos no funcionales

ID	Característica	Descripción y cómo se cumple
RNF01	Usabilidad	La interfaz Swing se organiza por módulos, valida los datos ingresados y muestra mensajes claros al usuario.
RNF02	Integridad	Las operaciones que modifican varias tablas se ejecutan en una transacción (commit o rollback) y la base activa las llaves foráneas y las restricciones CHECK.
RNF03	Seguridad (datos)	Todas las consultas con datos del usuario usan PreparedStatement para evitar inyección SQL (OWASP Foundation, s. f.).
RNF04	Seguridad (acceso)	Las opciones disponibles dependen del rol autenticado (Administrador o Consultor). Las contraseñas se almacenan como hashes SHA-256 y nunca en texto plano (sección 7.5).
RNF05	Mantenibilidad	El código se separa en paquetes model, view, controller y dao, con una clase por entidad y un DAO por tabla.
RNF06	Portabilidad	La aplicación corre en cualquier sistema con Java y usa SQLite embebido, sin instalar un servidor de base de datos.
RNF07	Rendimiento	Las consultas del historial usan índices por fecha y por grabador; se espera una respuesta inferior a 2 segundos con el volumen del caso de estudio.
RNF08	Trazabilidad	Todo cambio de stock o de estado genera un movimiento con usuario, fecha, valores anteriores y nuevos.

Nota. Elaboración propia.

5.3 Trazabilidad de requerimientos a la implementación

La Tabla 5 relaciona cada requerimiento con el panel de la capa Vista y el DAO que lo implementa. Esta correspondencia permite verificar que cada requerimiento funcional tiene código asociado.

Tabla 5

Trazabilidad entre requerimientos funcionales y código

Requerimiento	Vista (paquete view)	DAO (paquete dao)
RF01, RF12	GrabadorPanel	GrabadorExternoDAO.insertarConInventarioInicial; MovimientoInventarioDAO.registrarInicialEnTransaccion
RF02, RF03	GrabadorPanel	GrabadorExternoDAO.listarTodos, buscarPorId
RF04, RF05	GrabadorPanel	GrabadorExternoDAO.actualizar, desactivar
RF06	GrabadorPanel	GrabadorExternoDAO.buscarFiltrado, buscarGeneral
RF07	ComparadorPanel	GrabadorExternoDAO.buscarPorId, listarTodos
RF08	CamaraPanel	CamaraDAO
RF09	CompatibilidadPanel	CompatibilidadDAO
RF10	AccesorioPanel	AccesorioDAO
RF11	ImagenesGrabadorDialog	ImagenGrabadorDAO
RF13, RF14	InventarioPanel	MovimientoInventarioDAO.registrar, actualizarInventario; InventarioDAO
RF15	InventarioPanel	MovimientoInventarioDAO.listarEntreFechas
RF16	ReportePanel	ReporteDAO
RF17, RF19	LoginFrame, MainFrame	UsuarioDAO.buscarPorCorreo; PasswordUtil.verifyPassword; Usuario.verificarPermiso
RF18	UsuarioPanel	UsuarioDAO

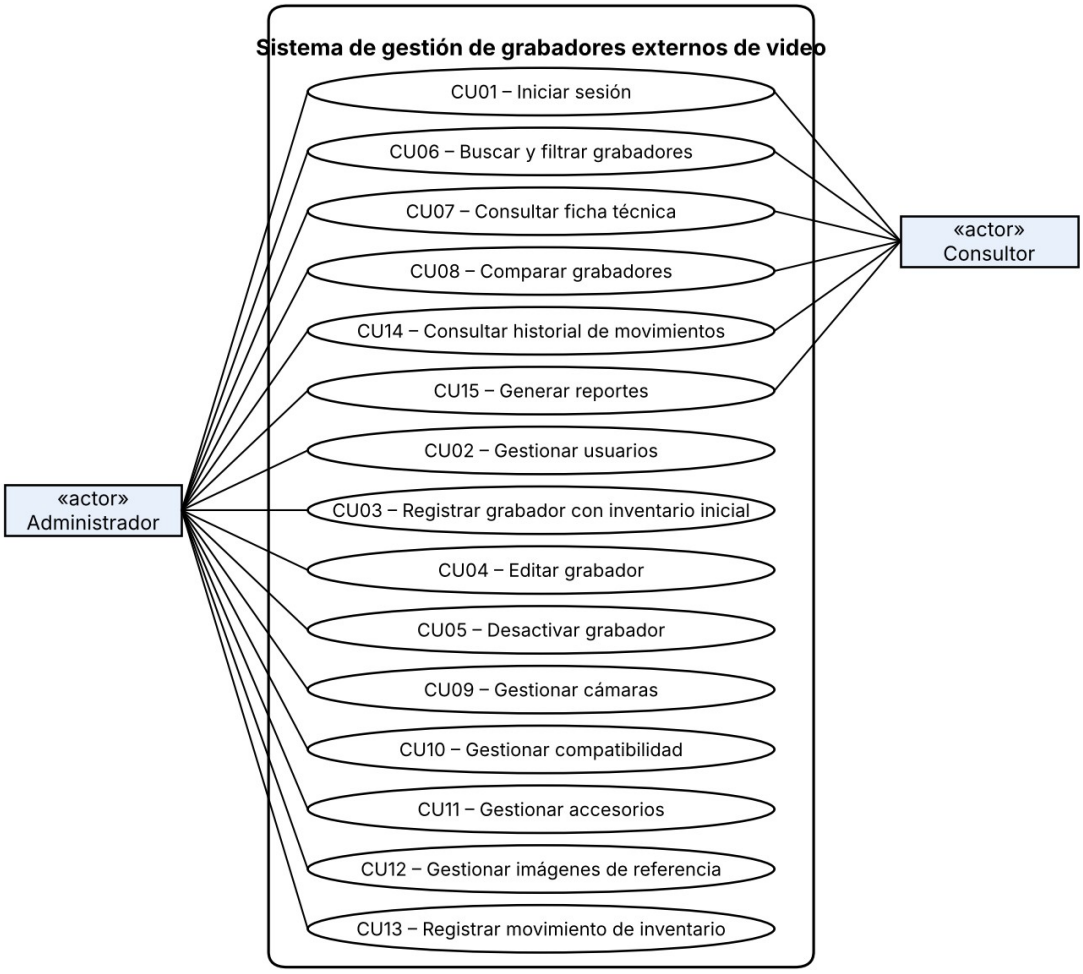
Nota. Elaboración propia.

6. Diagrama de casos de uso

El sistema tiene dos actores. El Administrador tiene acceso completo; el Consultor tiene acceso de solo lectura. Ambos son especializaciones del usuario autenticado, por lo que comparten el caso de uso Iniciar sesión.

Figura 1

Diagrama de casos de uso del sistema



Nota. Elaboración propia.

Tabla 6

Descripción de los casos de uso

Caso de uso	Actor	Descripción	RF
CU01 Iniciar sesión	A, C	Valida correo y contraseña y abre la ventana principal con los módulos del rol.	RF17, RF19

Caso de uso	Actor	Descripción	RF
CU02 Gestionar usuarios	A	Crea, edita y desactiva cuentas y asigna roles.	RF18
CU03 Registrar grabador con inventario inicial	A	Registra el equipo y su stock inicial en una sola transacción.	RF01, RF12
CU04 Editar grabador	A	Modifica los datos técnicos de un grabador.	RF04
CU05 Desactivar grabador	A	Marca el grabador como inactivo sin borrar su historial.	RF05
CU06 Buscar y filtrar grabadores	A, C	Consulta el listado con filtros y términos de búsqueda.	RF02, RF06
CU07 Consultar ficha técnica	A, C	Muestra los datos, imágenes, accesorios y compatibilidades.	RF03
CU08 Comparar grabadores	A, C	Contrasta las especificaciones de varios equipos.	RF07
CU09 Gestionar cámaras	A	Mantiene el catálogo de cámaras.	RF08
CU10 Gestionar compatibilidad	A	Relaciona grabadores y cámaras con un nivel de compatibilidad.	RF09
CU11 Gestionar accesorios	A	Mantiene los accesorios de cada grabador.	RF10
CU12 Gestionar imágenes de referencia	A	Asocia las vistas frontal, posterior y lateral.	RF11
CU13 Registrar movimiento de inventario	A	Registra entradas, salidas, ajustes y cambios de estado.	RF13, RF14
CU14 Consultar historial de movimientos	A, C	Lista movimientos filtrados por fecha, tipo y usuario.	RF15
CU15 Generar reportes	A, C	Genera reportes con los datos almacenados.	RF16

Nota. Elaboración propia.

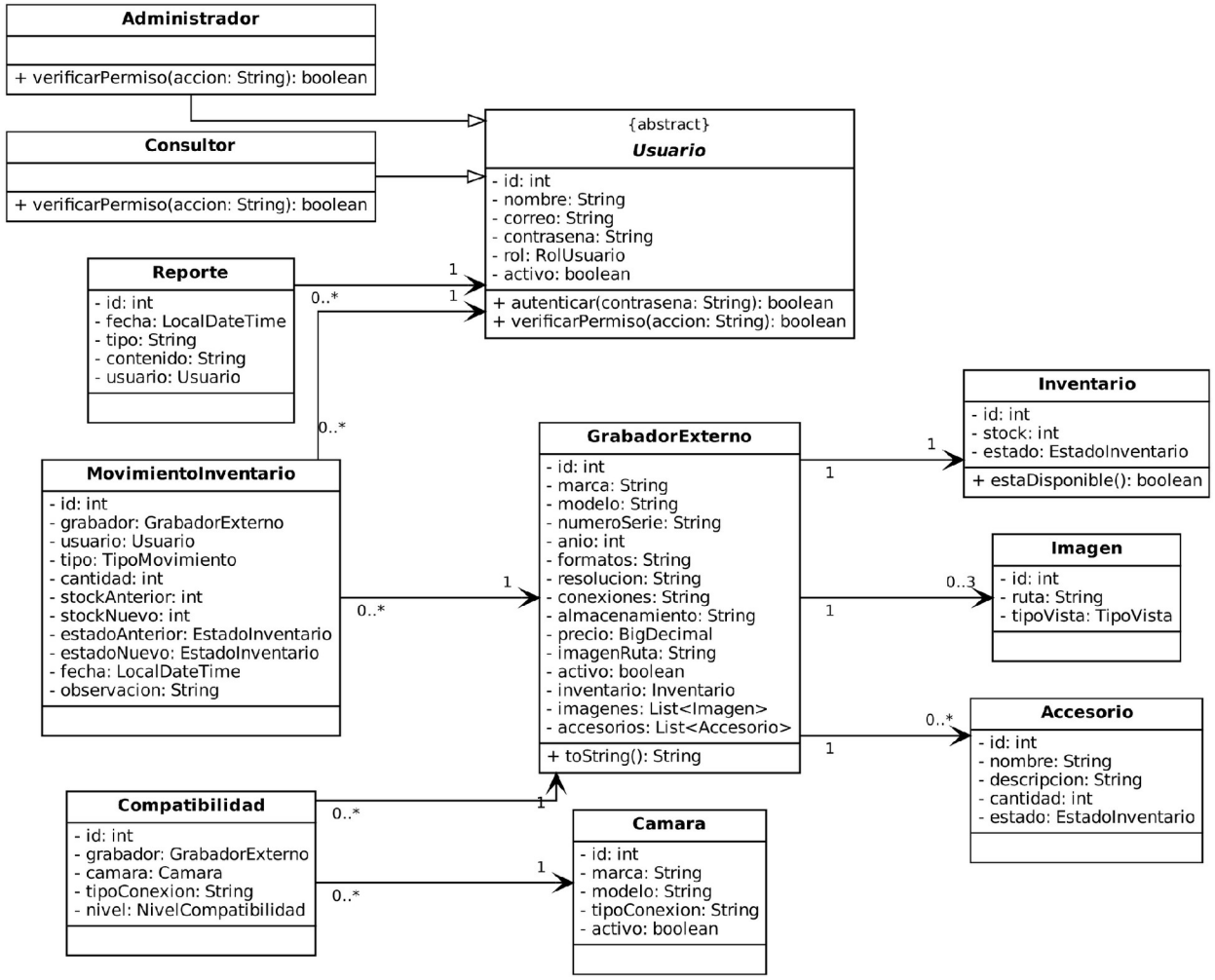
7. Diagrama de clases

7.1 Diagrama de clases del sistema

La Figura 2 presenta el diagrama de clases en notación UML (Object Management Group [OMG], 2017) ya corregido según las observaciones del docente. La clase abstracta Usuario es la superclase; Administrador y Consultor la especializan, y el triángulo de la herencia queda del lado de Usuario. Las asociaciones se expresan con referencias a objetos (por ejemplo, el atributo grabador de tipo GrabadorExterno en MovimientoInventario) y con colecciones (List<Imagen> en GrabadorExterno), no con llaves foráneas numéricas. Por claridad, no se dibujan los métodos get y set de cada atributo.

Figura 2

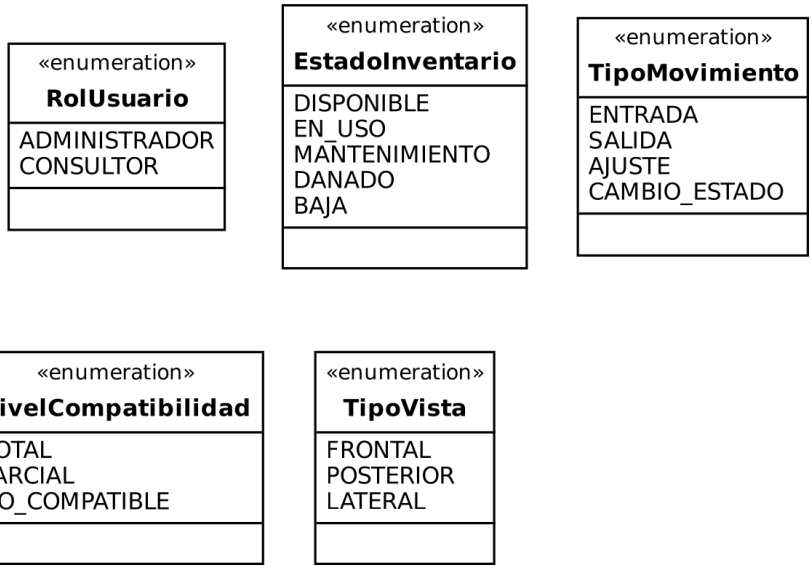
Diagrama de clases del sistema de gestión de grabadores externos de video



Nota. Elaboración propia.

Figura 3

Enumeraciones del dominio



Nota. Elaboración propia.

7.2 Descripción de las clases

Tabla 7

Clases del modelo y su responsabilidad

Clase	Responsabilidad
Usuario (abstracta)	Concentra los datos comunes de las cuentas y declara verificarPermiso(accion), que cada subclase implementa de forma distinta.
Administrador	Permite todas las acciones, incluidas las de registro, edición, inventario y administración de usuarios.
Consultor	Permite únicamente acciones de consulta, sin necesidad de condicionales sobre un atributo de tipo.
GrabadorExterno	Ficha técnica del equipo; agrega su Inventario, sus Imagen y sus Accesorio.
Inventario	Stock y estado del grabador; estaDisponible() devuelve verdadero si el stock es mayor que cero y el estado es DISPONIBLE.
MovimientoInventario	Registro auditable de cada cambio de stock o estado, con usuario y fecha.
Imagen	Ruta de una imagen de referencia y su vista (FRONTAL, POSTERIOR o LATERAL).
Accesorio	Elemento asociado a un grabador, con cantidad y estado.
Camara	Cámara con la que puede usarse un grabador.
Compatibilidad	Clase asociativa entre GrabadorExterno y Camara con el tipo de conexión y el nivel de compatibilidad.

Clase	Responsabilidad
Reporte	Reporte generado por un usuario en una fecha.

Nota. Elaboración propia.

La herencia se justifica porque Administrador y Consultor comparten los atributos de Usuario pero difieren en su comportamiento. El polimorfismo aparece cuando la ventana principal llama a verificarPermiso(accion) sobre un objeto Usuario y se ejecuta la versión de la subclase correspondiente, sin usar condicionales por rol (Deitel y Deitel, 2017; Larman, 2004). Además, las enumeraciones reemplazan las cadenas libres para estados y tipos, de modo que el compilador impide valores inválidos, y BigDecimal evita los errores de redondeo de double al manejar valores monetarios (Bloch, 2018).

7.3 Atención a las observaciones del docente

Tabla 8

Observaciones de la primera entrega y decisión tomada

Observación del docente	Decisión en esta entrega
Voltear las flechas de herencia: el triángulo va en Usuario.	Administrador y Consultor apuntan a Usuario con el triángulo vacío en la superclase (Figura 2).
Cambiar grabadorId, usuarioId y camaraId por referencias a objetos y listas, porque en Java no se guardan llaves foráneas.	Se usan atributos como grabador: GrabadorExterno y usuario: Usuario, y colecciones como List<Imagen>. Las llaves foráneas permanecen solo en las tablas; los DAO resuelven la conversión.
Usar enums para estados y tipos, LocalDate y BigDecimal.	Se definieron RolUsuario, EstadoInventario, TipoMovimiento, NivelCompatibilidad y TipoVista. Las fechas se manejan con LocalDateTime y el precio con BigDecimal.
Sacar registrar() y editar() de las entidades y llevarlos a los DAO.	Las entidades solo conservan comportamiento propio del dominio (verificarPermiso, estaDisponible). El registro y la edición están en los DAO (sección 7.5).
Decidir si GrabadorExterno es un modelo con stock o una unidad con número de serie.	Se define como modelo (ficha técnica) cuyo stock se lleva en Inventario. El número de serie se conserva como código único de identificación del registro del catálogo. El seguimiento unitario por serie queda como trabajo futuro mediante una clase Unidad.
Agregar requerimientos funcionales numerados, numerar páginas, entregar en PDF y ubicar el proyecto en un caso real con datos.	Se agregó el capítulo 5, el documento tiene páginas numeradas, se entrega en PDF y la sección 1.1 plantea el problema en el contexto de la industria audiovisual y de las plataformas existentes; el guion de la entrevista del caso de estudio se presenta en el Anexo A.
(Ajuste propio) La clase HistorialEstado de la primera entrega.	Se eliminó porque MovimientoInventario ya guarda el estado anterior y el nuevo mediante el tipo CAMBIO_ESTADO, lo que evita duplicar información.

Nota. Elaboración propia.

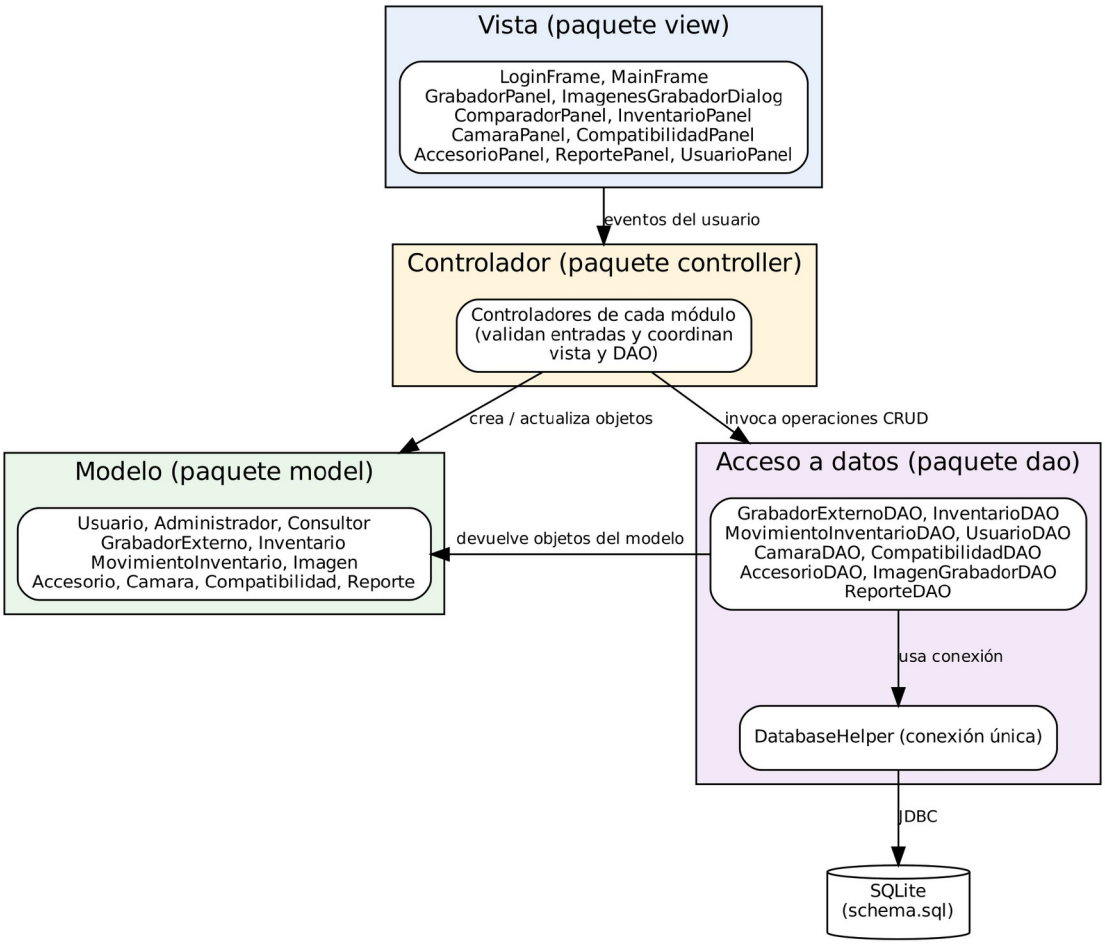
7.4 Arquitectura MVC

La aplicación separa responsabilidades en cuatro paquetes: view (paneles y ventanas Swing), controller (coordina la vista con el modelo y los DAO), model (clases del dominio) y dao (acceso a datos), siguiendo el patrón Modelo-Vista-Controlador y el patrón DAO (Gamma et al., 1994; Fowler, 2002). La vista no contiene sentencias SQL: solo captura eventos y muestra resultados; el SQL vive exclusivamente en los DAO, de modo que cada clase tiene una responsabilidad clara (Martin, 2008).

Por ejemplo, UsuarioController recibe los datos capturados en la vista, los valida con ValidatorUtil, aplica el hash con PasswordUtil y delega la persistencia en UsuarioDAO. Al terminar, devuelve a la vista un objeto Usuario (en el inicio de sesión) o un mensaje de resultado, "OK" o la descripción del error (en el registro y la actualización), de modo que la vista nunca consulta la base de datos directamente.

Figura 4

Arquitectura MVC de la aplicación por paquetes



Nota. Elaboración propia.

7.5 Implementación de las clases, la conexión y los DAO

Cada DAO obtiene la conexión de DatabaseHelper, que expone una única instancia mediante el método getInstance() (patrón Singleton). Así, todos los DAO comparten la misma conexión y el código de conexión no se repite (Gamma et al., 1994). El esquema activa las llaves foráneas con PRAGMA foreign_keys = ON, pues SQLite no las aplica por defecto (SQLite, s. f.-b).

Las sentencias CRUD usan PreparedStatement con parámetros, nunca concatenan valores ingresados por el usuario (OWASP Foundation, s. f.). La eliminación es lógica: desactivar(id) actualiza el campo activo y conserva los registros históricos. La búsqueda con filtros

(buscarFiltrado) arma la sentencia de forma dinámica agregando condiciones solo para los filtros diligenciados y pasando todos los valores como parámetros.

Las contraseñas se almacenan como hashes SHA-256, no en texto plano. Durante el inicio de sesión, el sistema calcula el hash de la contraseña ingresada y lo compara con el almacenado en SQLite. Esta lógica está encapsulada en la clase PasswordUtil (métodos hashPassword y verifyPassword), que es invocada por UsuarioController al iniciar sesión, al registrar y al actualizar usuarios; de esta forma, UsuarioDAO solo recibe y guarda el hash.

Las operaciones de inventario modifican más de una tabla, por lo que se ejecutan en una transacción. El método enTransaccion de MovimientoInventarioDAO recibe la operación como una interfaz funcional, desactiva el autocommit, confirma si el resultado es "OK", revierte en caso contrario y restaura el autocommit al final. El Código 1 muestra el método.

Código 1

Manejo de transacciones en MovimientoInventarioDAO

```
private String enTransaccion(SqlOperation operation) {
    boolean autoCommit = true;
    try {
        autoCommit = connection.getAutoCommit();
        connection.setAutoCommit(false);
        String result = operation.execute();
        if ("OK".equals(result)) connection.commit();
        else connection.rollback();
        return result;
    } catch (SQLException e) {
        try { connection.rollback(); }
        catch (SQLException rollbackError) { e.addSuppressed(rollbackError); }
        return "Error al registrar el movimiento de inventario.";
    } finally {
        try { connection.setAutoCommit(autoCommit); }
        catch (SQLException e) { /* se informa por consola */ }
    }
}
```

Nota. Fragmento abreviado del código fuente del proyecto.

El método registrar valida que la cantidad sea mayor que cero en entradas y salidas, que el tipo sea válido y que el stock resultante no sea negativo antes de actualizar la tabla Inventario e insertar el movimiento. El registro de un grabador nuevo (insertarConInventarioInicial en GrabadorExternoDAO) inserta el grabador y su inventario inicial en una sola transacción: si el número de serie está duplicado o falla el inventario, se revierte todo.

7.6 Interfaz gráfica de usuario

La interfaz usa Java Swing (Oracle, s. f.). LoginFrame autentica al usuario y MainFrame organiza los módulos en paneles. Cada panel combina tablas para listar datos, listas desplegables

para filtrar o seleccionar valores de catálogo, y campos y botones para registrar o editar. Los datos ingresados se validan antes de llamar al DAO y los errores se informan al usuario con mensajes.

Tabla 9

Componentes de la interfaz por módulo

Clase de la vista	Función	Componentes principales
LoginFrame	Inicio de sesión	Campos de correo y contraseña, botón Ingresar
MainFrame	Ventana principal y navegación	Pestañas o menú según el rol
GrabadorPanel	Registro, edición, desactivación y búsqueda de grabadores	Tabla de grabadores, listas de filtros, formulario
ImagenesGrabadorDialog	Imágenes frontal, posterior y lateral	Selector de archivo y vista previa
ComparadorPanel	Comparación de especificaciones	Listas para elegir equipos y tabla comparativa
InventarioPanel	Movimientos e historial	Lista de tipo y estado, tabla de historial, filtros por fecha
CamaraPanel	Catálogo de cámaras	Tabla y formulario
CompatibilidadPanel	Compatibilidad grabador-cámara	Listas de grabador, cámara y nivel; tabla
AccesorioPanel	Accesorios por grabador	Tabla y formulario
ReportePanel	Generación de reportes	Selección de tipo y tabla de resultados
UsuarioPanel	Administración de usuarios	Tabla, lista de rol y formulario

Nota. Elaboración propia.

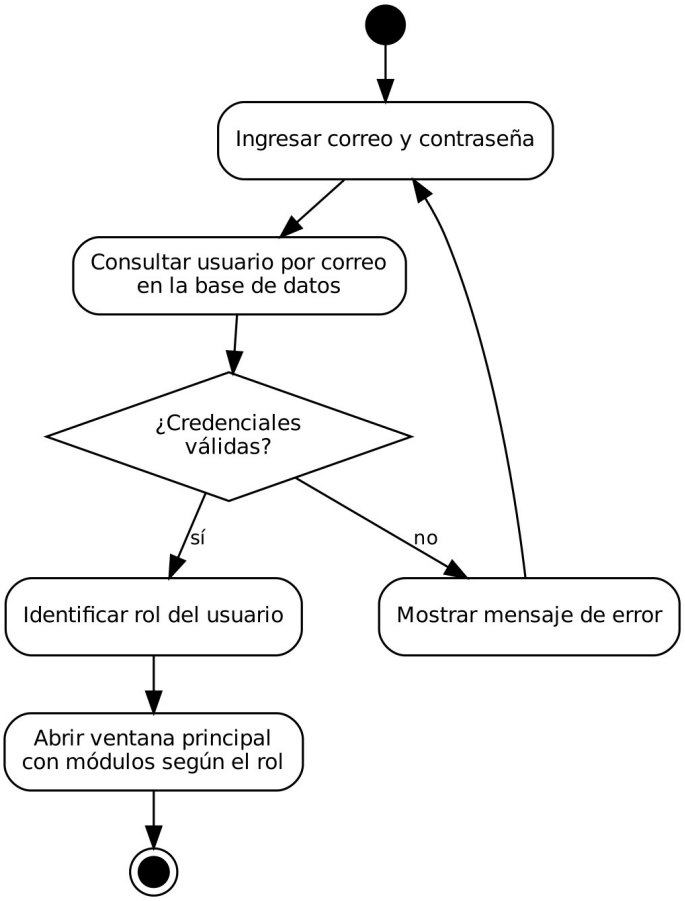
8. Diagramas de actividades

Se modelan tres procesos representativos: el inicio de sesión, el registro de un grabador con su inventario inicial y el registro de un movimiento de inventario.

8.1 Inicio de sesión

Figura 5

Diagrama de actividades del inicio de sesión



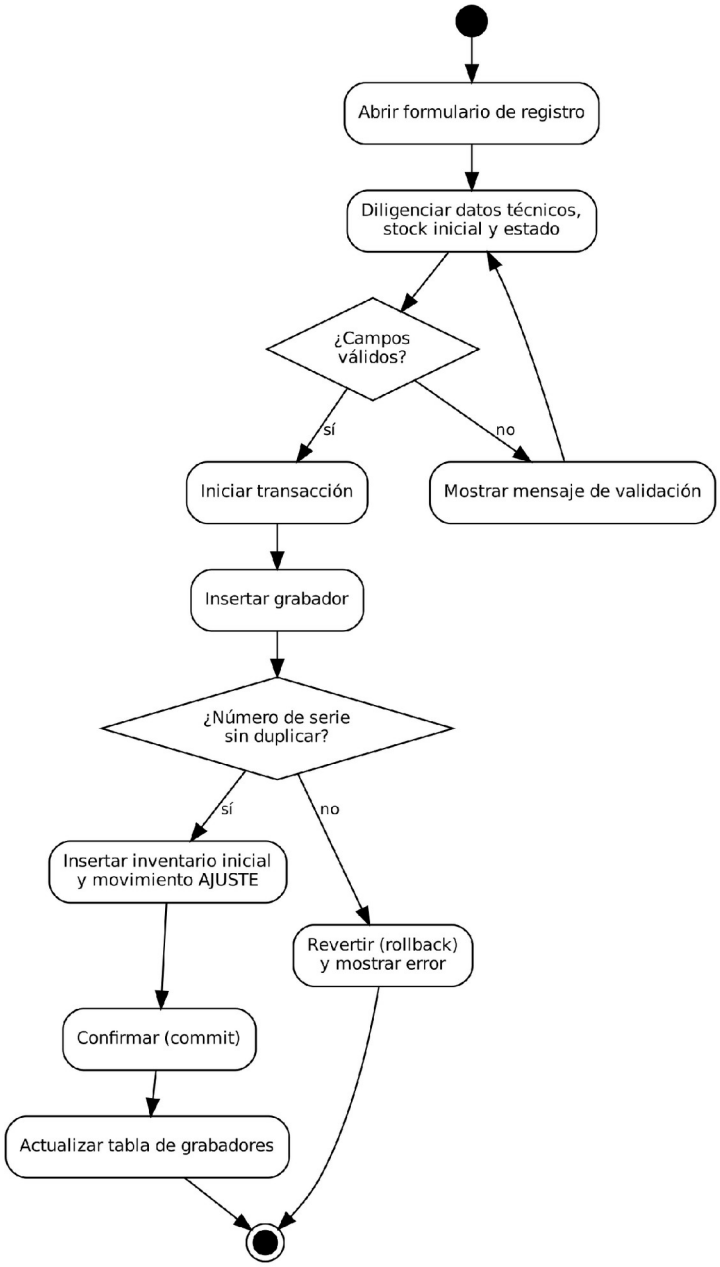
Nota. Elaboración propia.

8.2 Registro de un grabador con inventario inicial

Si los campos son válidos, se abre una transacción, se inserta el grabador y, si el número de serie no está duplicado, se crea el inventario inicial y su movimiento de tipo AJUSTE. Cualquier falla revierte la transacción.

Figura 6

Diagrama de actividades del registro de un grabador



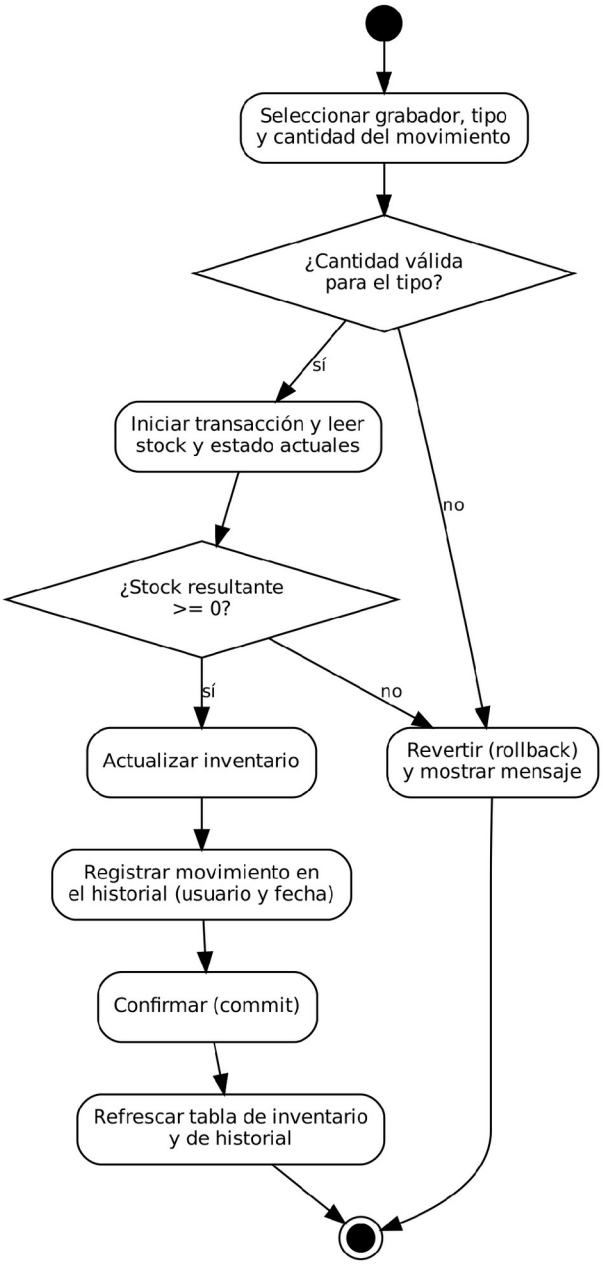
Nota. Elaboración propia.

8.3 Registro de un movimiento de inventario

Se leen el stock y el estado actuales, se calcula el stock resultante y, si no es negativo, se actualiza el inventario y se registra el movimiento en el historial; de lo contrario se revierte la operación.

Figura 7

Diagrama de actividades del registro de un movimiento de inventario



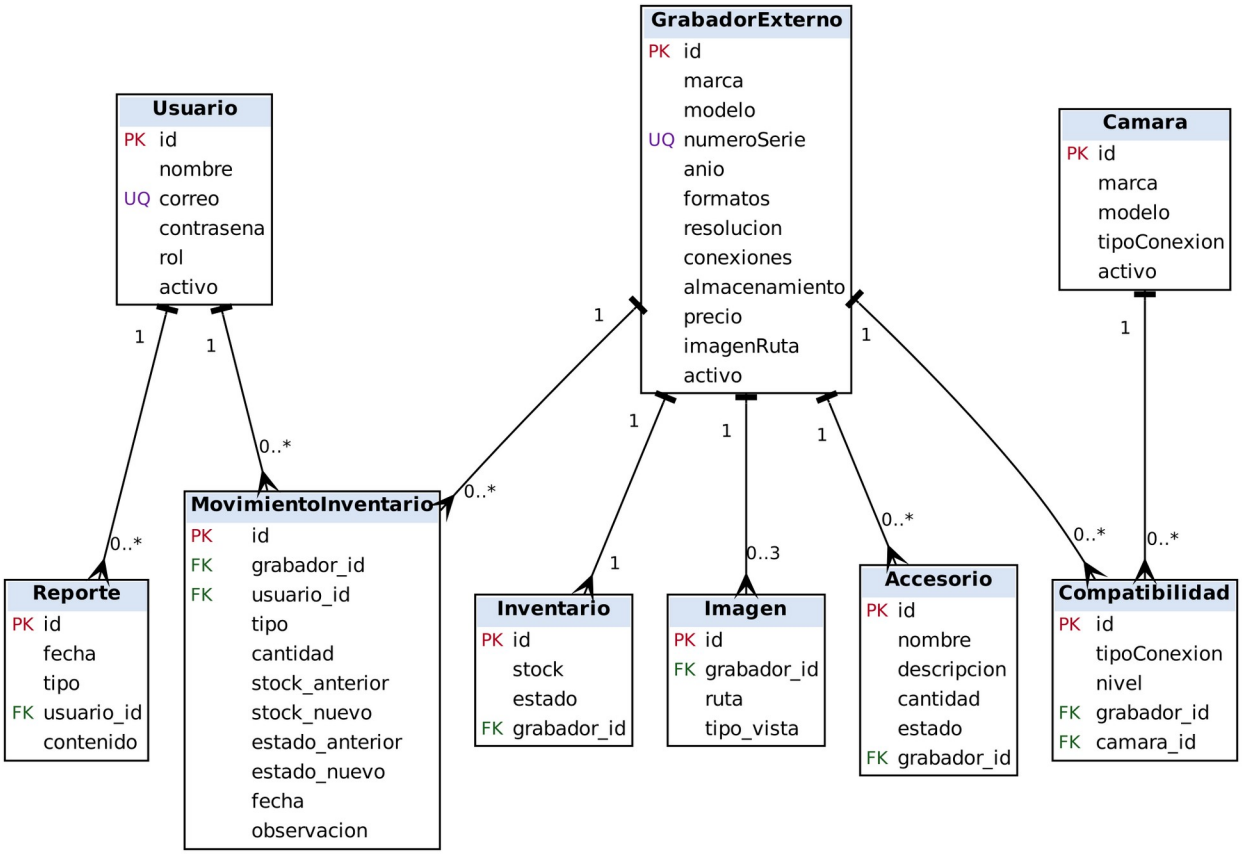
Nota. Elaboración propia.

9. Diagrama entidad-relación

El modelo tiene nueve entidades. GrabadorExterno es la entidad central: se relaciona uno a muchos con Imagen (máximo tres vistas), Accesorio, MovimientoInventario y Compatibilidad, y uno a uno con Inventario. Compatibilidad resuelve la relación muchos a muchos entre GrabadorExterno y Camara y guarda los atributos de la relación (tipo de conexión y nivel). Usuario se relaciona uno a muchos con MovimientoInventario y con Reporte, lo que permite saber quién hizo cada cambio (Date, 2004; Elmasri y Navathe, 2016).

Figura 8

Diagrama entidad-relación del sistema



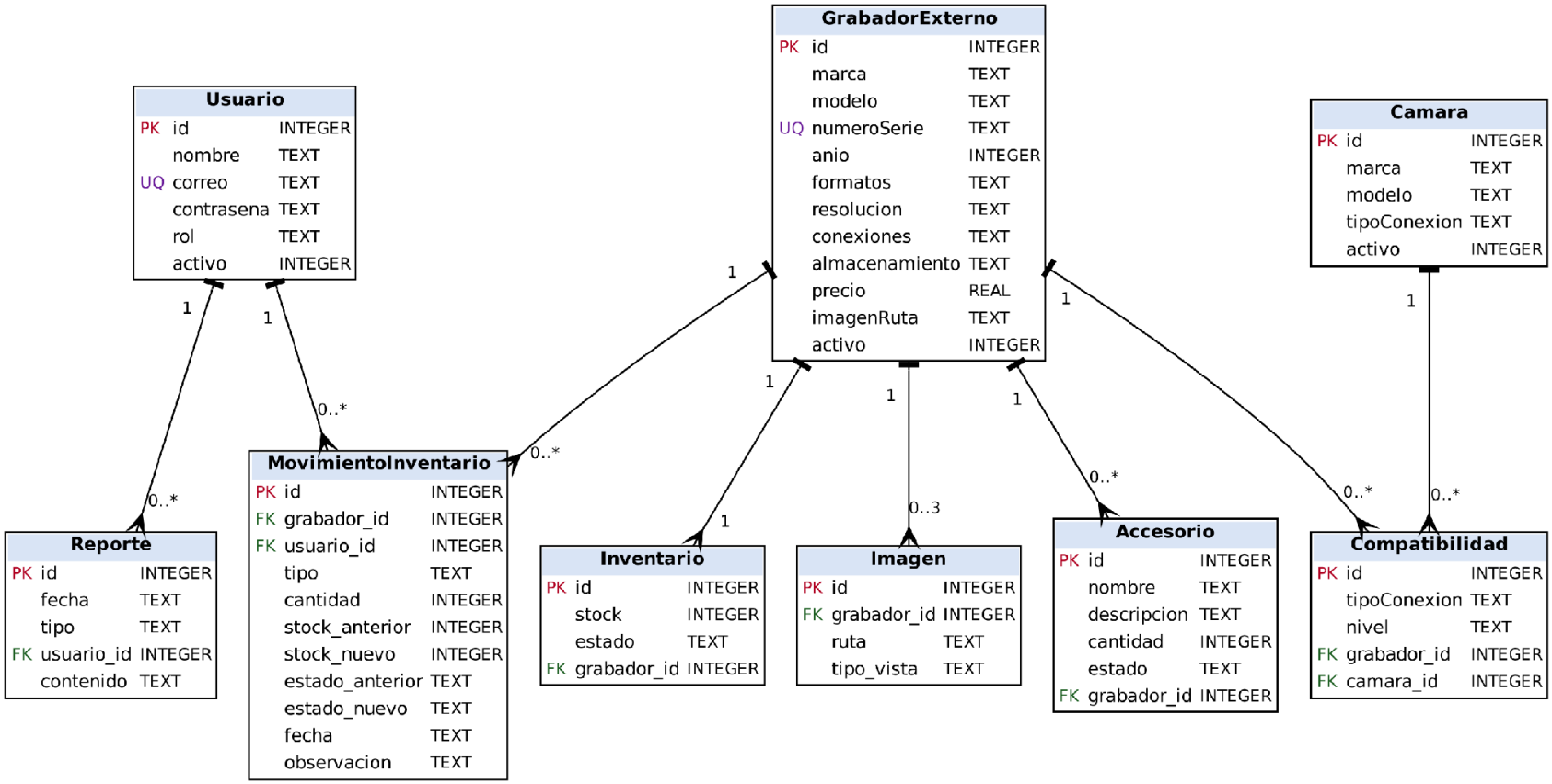
Nota. Elaboración propia.

La herencia Administrador y Consultor se representa en la entidad Usuario con el atributo discriminador rol (tabla única por jerarquía). La relación uno a uno entre GrabadorExterno e Inventario se garantiza en la base de datos mediante la restricción UNIQUE sobre grabador_id en la tabla Inventario, y los DAO la refuerzan al rechazar un segundo inventario para el mismo grabador.

10. Diagrama de datos

Figura 9

Diagrama de datos (modelo relacional) del sistema



Nota. Elaboración propia.

La Figura 9 muestra el modelo relacional implementado en SQLite con sus tipos de dato, llaves primarias (PK), foráneas (FK) y únicas (UQ). Las tablas están en tercera forma normal: cada atributo depende de la llave de su tabla, los datos repetibles (imágenes,

accesorios, movimientos, compatibilidades) están en tablas propias y no hay atributos derivados almacenados, salvo los valores anteriores en MovimientoInventario, que se guardan a propósito para la auditoría, y el campo imagenRuta de GrabadorExterno, que conserva la ruta de la imagen principal del equipo como acceso directo para los listados y constituye una redundancia controlada respecto de la tabla Imagen, donde se gestionan las tres vistas de referencia (Elmasri y Navathe, 2016).

Tabla 10

Restricciones de integridad de la base de datos

Tabla	Restricciones
Usuario	correo UNIQUE; rol CHECK en (administrador, consultor); activo con valor por defecto 1; contrasena almacenada como hash SHA-256.
GrabadorExterno	numeroSerie UNIQUE; activo con valor por defecto 1 (eliminación lógica).
Inventario	stock CHECK (stock >= 0); estado CHECK en (disponible, en_uso, mantenimiento, danado, baja); grabador_id UNIQUE (garantiza la relación 1:1 con el grabador) y FK a GrabadorExterno.
Imagen	tipo_vista CHECK en (FRONTAL, POSTERIOR, LATERAL); UNIQUE(grabador_id, tipo_vista), lo que limita a tres imágenes por grabador.
MovimientoInventario	tipo CHECK en (ENTRADA, SALIDA, AJUSTE, CAMBIO_ESTADO); cantidad, stock_anterior y stock_nuevo mayores o iguales a cero; índices por fecha y por grabador y fecha.
Compatibilidad	nivel CHECK en (total, parcial, no_compatible); UNIQUE(grabador_id, camara_id).
Reporte, Accesorio	FK a Usuario y a GrabadorExterno respectivamente.

Nota. Elaboración propia.

SQLite almacena las fechas como texto y los booleanos como enteros. Por eso la capa DAO convierte el texto de fecha a LocalDateTime, el campo activo (0 o 1) a boolean, el texto de estado y tipo a los enums de Java y el precio a BigDecimal. El script SQL completo se presenta en el Anexo B.

11. Mockup

Las maquetas muestran la organización de las pantallas principales. La ventana de inicio de sesión valida las credenciales; la ventana principal agrupa los módulos en pestañas, y las pantallas de grabadores e inventario combinan filtros, listas desplegables y tablas.

Figura 10

Maqueta de la ventana de inicio de sesión



Nota. Elaboración propia.

Figura 11

Maqueta del módulo de grabadores

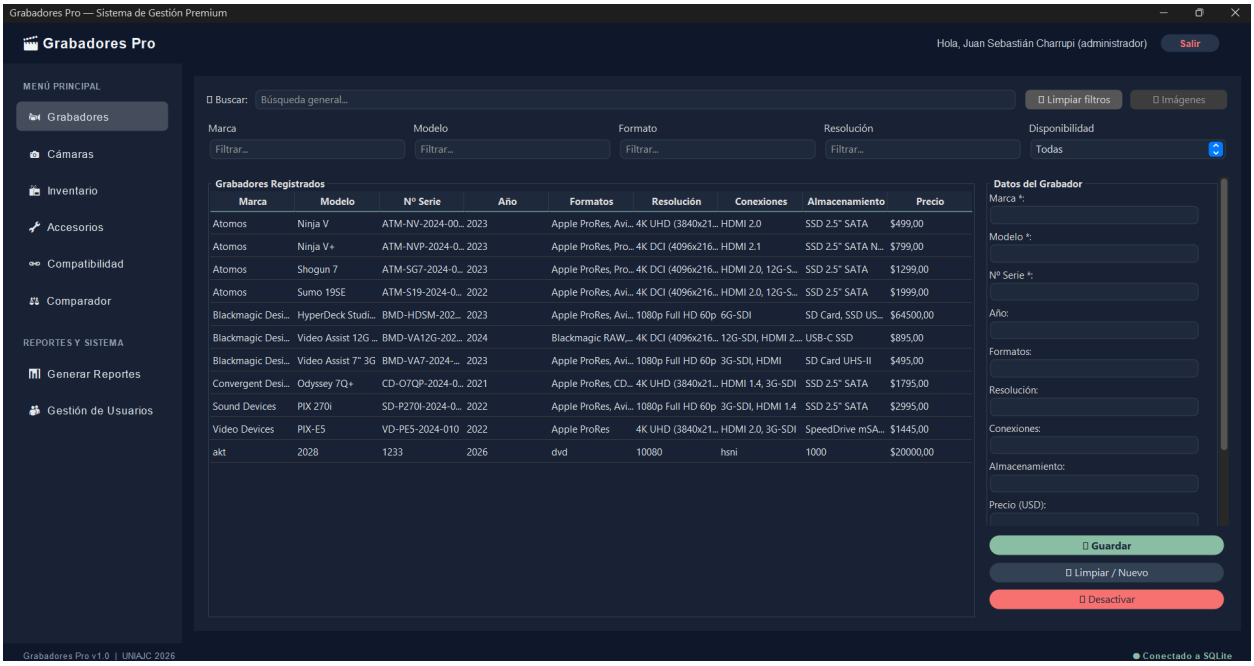
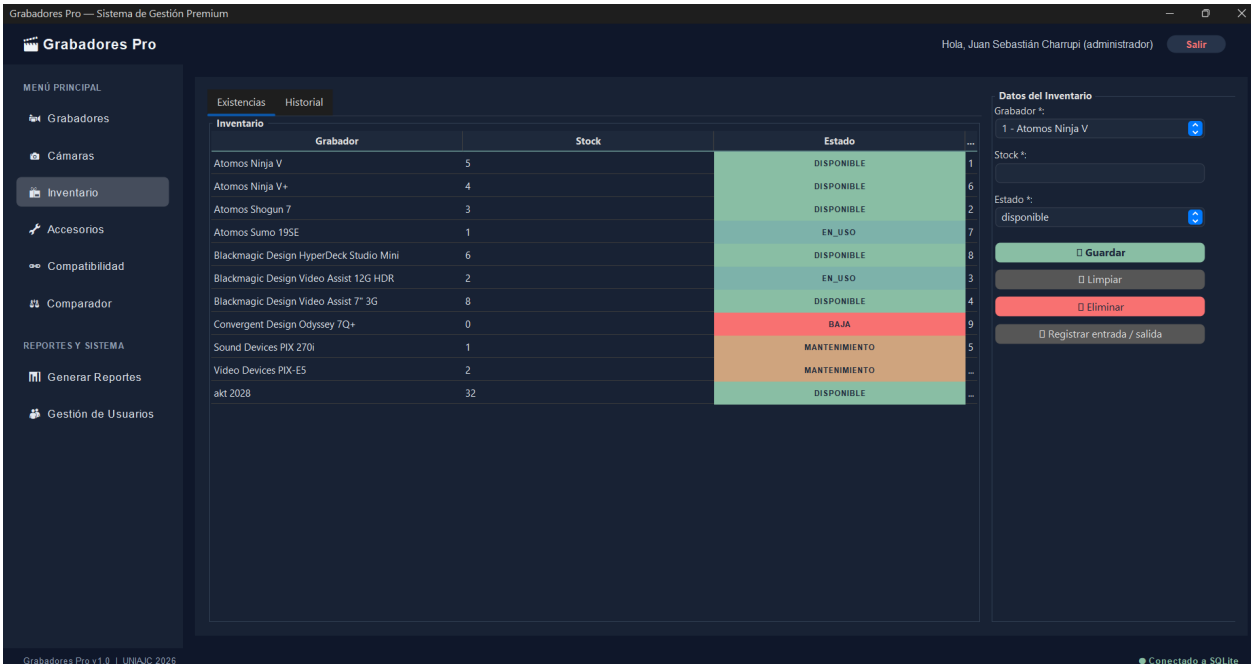


Figura 12

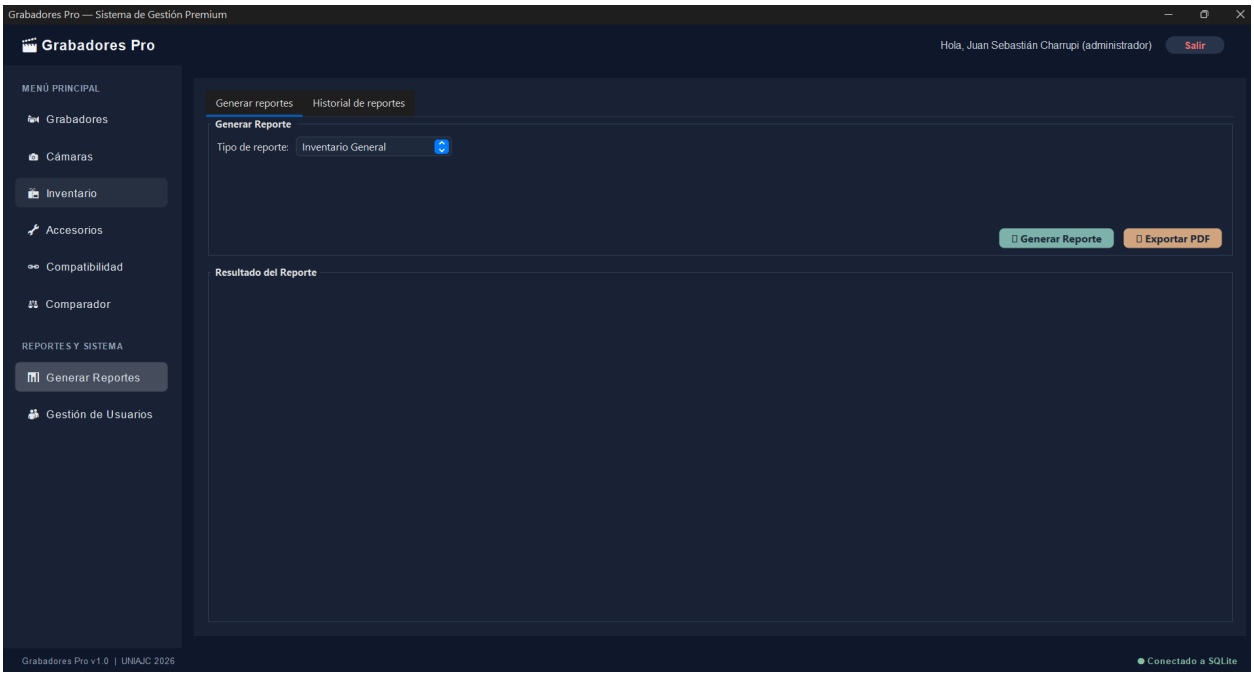
Maqueta del módulo de inventario y movimientos

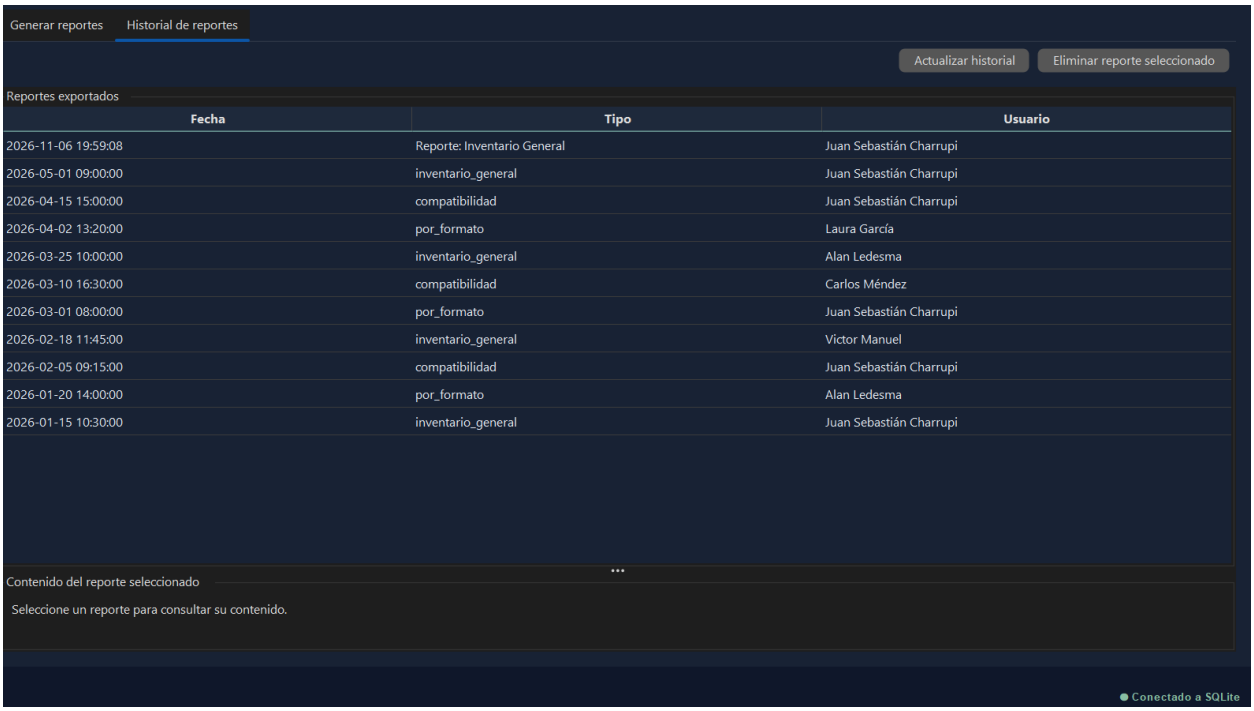


Nota. Elaboración propia.

Figura 13

Maqueta del módulo de generación de reportes e historial de reportes





Nota. Elaboración propia.

Referencias

Atomos. (s. f.). *About us*. https://www.atomos.com/about/

Blackmagic Design. (s. f.). *Blackmagic Video Assist*. https://www.blackmagicdesign.com/products/blackmagicvideoassist

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Date, C. J. (2004). *Introducción a los sistemas de bases de datos* (8.ª ed.). Pearson Educación.

Deitel, P., y Deitel, H. (2017). *Java: Cómo programar* (10.ª ed.). Pearson Educación.

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

International Organization for Standardization [ISO], e International Electrotechnical Commission [IEC]. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Object Management Group [OMG]. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1* (formal/17-12-05). https://www.omg.org/spec/UML/2.5.1/

Oracle. (s. f.). *Creating a GUI with Swing*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/

OWASP Foundation. (s. f.). *SQL injection prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Software engineering* (9.ª ed.). Pearson.

SQLite. (s. f.-a). *SQLite documentation*. https://www.sqlite.org/docs.html

SQLite. (s. f.-b). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

Video Devices. (s. f.). *Video Devices* [Sitio web oficial]. https://www.videodevices.com

Anexos

Anexo A. Guion de entrevista para el levantamiento de requerimientos

Guion propuesto para la entrevista del caso de estudio.

1. ¿Cuántos grabadores externos de video maneja la organización y de qué marcas y modelos?

   :La cantidad de grabadores depende de la organización y de su inventario actual. Se contempla gestionar equipos de diferentes marcas y modelos, como Atomos, Blackmagic Design y Video Devices, que son fabricantes de referencia para el proyecto.

2. ¿Cómo registra hoy la información de cada equipo (hoja de cálculo, cuaderno, otro)?
   La información se encuentra distribuida en diferentes fuentes, como hojas de cálculo, documentos, catálogos de fabricantes y registros manuales. Esto dificulta mantener centralizadas y actualizadas las especificaciones técnicas de cada equipo.

3. ¿Cómo sabe cuántas unidades están disponibles, en uso o en mantenimiento?
   Actualmente, esta información puede requerir una verificación manual. El sistema propuesto permitirá consultar el inventario mediante los estados disponible, en uso, mantenimiento, dañado y baja, además de mostrar las cantidades registradas.
4. ¿Cómo verifica si un grabador es compatible con una cámara antes de una producción?
   Se consultan las especificaciones técnicas de los grabadores y las cámaras, especialmente sus conexiones y formatos compatibles. La aplicación permitirá registrar esta relación e identificar el nivel de compatibilidad como total, parcial o no compatible.
5. ¿Ha tenido problemas por no encontrar un equipo o por un dato desactualizado? Describa un caso.
   La falta de centralización puede ocasionar dificultades para localizar equipos, consultar sus características o conocer su disponibilidad real. Por ejemplo, podría aparecer un grabador como disponible en un registro, cuando realmente se encuentra en uso o mantenimiento.

6. ¿Qué accesorios acompañan a cada grabador y cómo se controlan?

   Los grabadores pueden contar con accesorios como baterías, cables, adaptadores, soportes y discos de almacenamiento. Se propone registrarlos individualmente en el sistema, indicando su nombre, descripción, cantidad y estado, y asociándolos al grabador correspondiente.

7. ¿Quiénes consultan y quiénes modifican la información? ¿Qué debería poder hacer cada tipo de usuario?

   Se contemplan dos tipos de usuarios:

- **Administrador: puede registrar, consultar, editar y desactivar grabadores, gestionar cámaras, accesorios, compatibilidades, inventario, movimientos, reportes y usuarios.**
- **Consultor: puede consultar la información de los grabadores, revisar fichas técnicas, comparar equipos, consultar compatibilidades, historial de movimientos y generar reportes, sin modificar los registros.**

8. ¿Qué reportes necesita para tomar decisiones de compra, mantenimiento o préstamo?

   Se necesitan reportes del inventario general, disponibilidad de grabadores, equipos en mantenimiento o dañados, movimientos de entrada y salida, historial de cambios, accesorios asociados y especificaciones técnicas. Esta información facilita tomar decisiones sobre adquisición, mantenimiento y asignación de equipos.

9. ¿Qué funciones considera indispensables en una aplicación para resolver estos problemas?

   La aplicación debe permitir registrar, consultar, editar y desactivar grabadores; gestionar cámaras y accesorios; verificar compatibilidades; controlar el inventario; registrar entradas, salidas, ajustes y cambios de estado; consultar el historial de movimientos; comparar especificaciones técnicas; administrar usuarios con diferentes permisos y generar reportes.

Anexo B. Script SQL de la base de datos

```
-- ========================================
-- SCHEMA SQLite — Grabadores Externos de Video
```

```
-- Proyecto Integrador III - UNIAJC
-- ========================================

PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;

-- TABLA: Usuario
CREATE TABLE IF NOT EXISTS Usuario (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre      TEXT    NOT NULL,
    correo      TEXT    NOT NULL UNIQUE,
    contrasena  TEXT    NOT NULL,  -- hash SHA-256 (hexadecimal)
    rol         TEXT    NOT NULL CHECK(rol IN ('administrador', 'consultor')),
    activo      INTEGER NOT NULL DEFAULT 1
);

-- TABLA: GrabadorExterno
CREATE TABLE IF NOT EXISTS GrabadorExterno (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    marca           TEXT    NOT NULL,
    modelo          TEXT    NOT NULL,
    numeroSerie     TEXT    NOT NULL UNIQUE,
    anio            INTEGER,
    formatos        TEXT,
    resolucion      TEXT,
    conexiones      TEXT,
    almacenamiento  TEXT,
    precio          REAL    DEFAULT 0.0,
    imagenRuta      TEXT,
    activo          INTEGER NOT NULL DEFAULT 1
);

-- TABLA: Camara
CREATE TABLE IF NOT EXISTS Camara (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    marca           TEXT    NOT NULL,
    modelo          TEXT    NOT NULL,
    tipoConexion    TEXT,
    activo          INTEGER NOT NULL DEFAULT 1
);

-- TABLA: Reporte
CREATE TABLE IF NOT EXISTS Reporte (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    fecha       TEXT    NOT NULL DEFAULT (datetime('now','localtime')),
    tipo        TEXT    NOT NULL,
    usuario_id  INTEGER NOT NULL,
    contenido   TEXT,
    FOREIGN KEY (usuario_id) REFERENCES Usuario(id)
);

-- TABLA: Inventario
CREATE TABLE IF NOT EXISTS Inventario (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    stock       INTEGER NOT NULL DEFAULT 0 CHECK(stock >= 0),
    estado      TEXT    NOT NULL CHECK(estado IN
('disponible','en_uso','mantenimiento','danado','baja')),
    grabador_id INTEGER NOT NULL UNIQUE,
    FOREIGN KEY (grabador_id) REFERENCES GrabadorExterno(id)
);

-- TABLA: Imágenes de referencia del grabador
CREATE TABLE IF NOT EXISTS Imagen (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    grabador_id     INTEGER NOT NULL,
    ruta            TEXT    NOT NULL,
    tipo_vista      TEXT    NOT NULL CHECK(tipo_vista IN ('FRONTAL','POSTERIOR','LATERAL')),
    UNIQUE(grabador_id, tipo_vista),
    FOREIGN KEY (grabador_id) REFERENCES GrabadorExterno(id)
);
```

```
-- TABLA: Historial auditable de movimientos de inventario
CREATE TABLE IF NOT EXISTS MovimientoInventario (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    grabador_id     INTEGER NOT NULL,
    usuario_id      INTEGER NOT NULL,
    tipo            TEXT    NOT NULL CHECK(tipo IN
('ENTRADA','SALIDA','AJUSTE','CAMBIO_ESTADO')),
    cantidad        INTEGER NOT NULL CHECK(cantidad >= 0),
    stock_anterior  INTEGER NOT NULL CHECK(stock_anterior >= 0),
    stock_nuevo     INTEGER NOT NULL CHECK(stock_nuevo >= 0),
    estado_anterior TEXT,
    estado_nuevo    TEXT,
    fecha           TEXT    NOT NULL DEFAULT (datetime('now','localtime')),
    observacion     TEXT,
    FOREIGN KEY (grabador_id) REFERENCES GrabadorExterno(id),
    FOREIGN KEY (usuario_id) REFERENCES Usuario(id)
);

CREATE INDEX IF NOT EXISTS idx_movimiento_fecha ON MovimientoInventario(fecha);
CREATE INDEX IF NOT EXISTS idx_movimiento_grabador ON MovimientoInventario(grabador_id, fecha);

-- TABLA: Accesorio
CREATE TABLE IF NOT EXISTS Accesorio (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre      TEXT    NOT NULL,
    descripcion TEXT,
    cantidad    INTEGER NOT NULL DEFAULT 0,
    estado      TEXT    NOT NULL DEFAULT 'disponible',
    grabador_id INTEGER NOT NULL,
    FOREIGN KEY (grabador_id) REFERENCES GrabadorExterno(id)
);

-- TABLA: Compatibilidad
CREATE TABLE IF NOT EXISTS Compatibilidad (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    tipoConexion    TEXT    NOT NULL,
    nivel           TEXT    NOT NULL CHECK(nivel IN ('total','parcial','no_compatible')),
    grabador_id     INTEGER NOT NULL,
    camara_id       INTEGER NOT NULL,
    FOREIGN KEY (grabador_id) REFERENCES GrabadorExterno(id),
    FOREIGN KEY (camara_id)   REFERENCES Camara(id),
    UNIQUE(grabador_id, camara_id)
);
```

Anexo C. Estructura de paquetes del proyecto

Tabla 11

Paquetes y archivos del código fuente

Paquete	Contenido
com.grabadores (raíz)	App.java: punto de entrada de la aplicación.
com.grabadores.model	Clases del dominio (entidades).
com.grabadores.view	LoginFrame, MainFrame, GrabadorPanel, ImagenesGrabadorDialog, ComparadorPanel, InventarioPanel, CamaraPanel, CompatibilidadPanel, AccesorioPanel, ReportePanel, UsuarioPanel.
com.grabadores.controller	UsuarioController y los demás controladores de cada módulo, que validan los datos de entrada, aplican la lógica de negocio y coordinan la vista con los DAO.

Paquete	Contenido
com.grabadores.dao	DatabaseHelper, GrabadorExternoDAO, InventarioDAO, MovimientoInventarioDAO, CamaraDAO, CompatibilidadDAO, AccesorioDAO, ImagenGrabadorDAO, ReporteDAO, UsuarioDAO.
com.grabadores.util	PasswordUtil (hash SHA-256 y verificación de contraseñas) y ValidatorUtil (validación de los datos de entrada).

Nota. Elaboración propia.